

Dentalspot

Estado del Proyecto y Roadmap Tecnico

Documento explicativo en lenguaje simple sobre los avances tecnicos del SaaS odontologico, lo pendiente, y respuestas a preguntas operativas concretas.

Fecha	18 de abril de 2026
Branch	main (sincronizado con origin/main)
Frente principal cubierto	Auditoria clinica + Consent + Hardening rol patient
Cobertura compliance	Ley 20.584 (consentimiento informado)
Cobertura compliance pendiente	Ley 21.719 (datos personales) - parcial

1. Resumen ejecutivo en una pagina

Dentalspot es una aplicacion web (SaaS - software como servicio) que cualquier dentista u odontologo en Chile puede usar para gestionar su clinica: pacientes, agenda, ficha clinica, odontograma, planes de tratamiento, evaluaciones, etc.

El trabajo realizado en estas sesiones se centro en **cerrar la base de seguridad y trazabilidad clinica**: que cada clinica vea solo sus pacientes, que cada acceso a una ficha quede grabado, que el consentimiento informado sea juridicamente real y no solo cosmetico, y que el paciente tenga control sobre su informacion. Ahora la app ya no miente al usuario (antes habian botones que decian 'Guardado' sin guardar nada) y la auditoria es real.

El resultado es una aplicacion **operativa, segura en lo basico y compliance-ready** para Ley 20.584. Falta cubrir Ley 21.719 (datos personales) en su totalidad, pulir bugs operativos pequenos, y agregar funcionalidades nuevas como pasarela de pago e idealmente herramientas de telemedicina.

Lo cubierto vs lo pendiente, en numeros:

Area	Estado	Detalle corto
Aislamiento por clinica (multi-tenant)	OK	Ya nadie ve pacientes de otra clinica
Auditoria clinica	OK 6/6 validado	Cada apertura de ficha y odontograma queda grabada
Consentimiento informado (Ley 20.584)	OK	Solo el paciente puede firmar; ya no se puede falsificar
Hardening rol patient	OK	Paciente no puede cambiar su propio diagnostico
Banner del dentista	OK	Ya muestra estado real de firma del paciente
Modal del dentista (firma falsa)	OK	Ahora es informativo, ya no falsifica
Bug routing modulo Odontograma	Pendiente	Sidebar y boton llevan a 404
Bug auditoria post-creacion evaluacion	Pendiente	Primera evaluacion no graba sin refresh
Schema drift (price, patient_id)	Pendiente	Algunas columnas viejas vs codigo nuevo
Compliance Ley 21.719 (ARCO)	Pendiente grande	consent_records y data_requests
Pasarela de pago	Pendiente nuevo	Tabla 'payments' existe pero sin integracion real
Telemedicina (foto + IA)	Idea	Factible con resguardos legales

2. Lo que mejoramos, explicado con manzanas

Cada bloque siguiente describe un area que se trabajo. Para que se entienda sin necesidad de saber codigo, cada uno tiene una analogia cotidiana antes del detalle tecnico.

2.1 Aislamiento por clinica (multi-tenant)

***Analogia:** Antes era como un cuaderno gigante donde TODAS las clinicas escribian sus pacientes en las mismas paginas, y cada dentista tenia que confiar en que solo leeria 'lo suyo'. Ahora cada clinica tiene SU PROPIO cuaderno con candado: aunque un dentista entre con la app, no puede ver los pacientes de la clinica de al lado, ni siquiera por accidente.*

Tecnicamente: cada paciente, cita y registro clinico tiene una columna **organization_id** que indica a que clinica pertenece. Las reglas de la base de datos (RLS, Row Level Security) verifican esa columna en cada consulta. Si un usuario intenta leer datos de otra clinica, la base de datos directamente no le devuelve nada. La seguridad esta en la base, no solo en el frontend (que cualquier programador podria saltarse).

2.2 Reglas de seguridad por rol (RLS)

***Analogia:** Imagina un edificio con varios pisos y un portero. La app es la puerta de entrada al edificio. RLS (Row Level Security) es el portero. Aunque un usuario tenga llave de la puerta principal, el portero le pregunta '?quien eres?' y solo le deja subir a los pisos donde puede estar. Un paciente puede subir solo a 'su' piso (sus datos). Un dentista puede subir a los pisos de los pacientes que atiende. Un admin puede entrar a todos los de su clinica.*

Tecnicamente: existen aproximadamente 99 policias (reglas) que controlan quien puede leer, escribir, actualizar o borrar cada tabla. Roles definidos: **dentist**, **clinic_admin**, **assistant**, **patient**, **platform_admin**. Por ejemplo: el rol 'patient' tiene policias que solo le permiten LEER su propia ficha; no puede escribir en datos clinicos.

2.3 Auditoria clinica (registro de accesos)

***Analogia:** Como una camara de seguridad en el pasillo de la clinica que graba cada vez que alguien entra a una sala de pacientes. Si despues hay un reclamo del estilo '?quien vio mi ficha el martes?', hay grabacion de quien, cuando, y a que ficha entro. La grabacion no se puede borrar.*

Tecnicamente: existe la tabla **clinical_audit_log** con triggers *append-only* (no se puede actualizar ni borrar filas). Cada vez que un dentista o asistente abre la ficha de un paciente o un odontograma, el frontend invoca un hook (**useClinicalAccessLogger**) que graba la accion: quien (**user_id**), a quien (**patient_id**), que recurso (**clinical_record** o **odontogram**), de que organizacion, en que momento. El paciente puede ver su propio historial de accesos en una pagina dedicada (**PatientAccessHistoryPage**).

Ademas hay un sistema de doble seguro contra spam: si el dentista entra y sale 5 veces en la misma hora, no se graban 5 lineas; se graba una. El usuario ve el detalle real (no inflado) y la base de datos no se llena de ruido.

2.4 Consentimiento informado (Ley 20.584)

***Analogia:** Antes era como si el dentista pudiera ir a la notaria y firmar EN NOMBRE del paciente sin que el paciente este. La firma quedaba registrada como si fuera del paciente, pero en realidad era del dentista. Eso es ilegal en la vida real y ahora tambien en la app: solo el paciente puede firmar su propio consentimiento, desde su propio celular o computador, con su propia cuenta.*

Tecnicamente: la firma se registra en la tabla **legal_signatures** con el **user_id** del paciente real, su user agent (navegador), su IP, y la version del documento firmado. El banner que el dentista ve sobre la ficha del paciente ya no consulta una columna vieja (**patients.clinical_consent_signed**) que el dentista podia manipular, sino que consulta una funcion en la base de datos llamada **get_patient_consent_status**, que mira **legal_signatures** y deriva el estado real. El modal que antes tenia un boton 'Firmar Consentimiento' fue convertido en informativo: ahora solo le explica al dentista que el paciente debe firmar desde su portal.

2.5 Hardening del rol paciente (que puede tocar el paciente)

***Analogia:** Antes el paciente, desde su portal, podia editar su propio diagnostico clinico (?cuantas caries tengo? '0'). Eso no tiene sentido medico ni legal. Ahora el paciente solo puede leer su informacion clinica. Para cambiar datos administrativos suyos (telefono, email) si puede; para cambiar diagnostico, alergias o antecedentes medicos NO.*

Tecnicamente: se creo un trigger en la base de datos (**prevent_patient_clinical_edit**) que, antes de aceptar un UPDATE sobre la tabla **patients**, verifica si el usuario es el dueño de la fila (es decir, el propio paciente). Si lo es, rechaza cualquier cambio en columnas clinicas (medical_history, diagnosis, allergies, medications, etc.) o en columnas de asignacion (organization_id, therapist_id). Si el usuario es el dentista o un admin, deja pasar normalmente. La UI del portal patient tambien se ajusto: los campos clinicos ya no se muestran como editables, evitando que el paciente apriete 'Guardar' y vea un toast falso de exito.

2.6 Banner y modal del dentista (honestidad de UI)

***Analogia:** Antes el banner amarillo en la ficha del paciente decia 'Consentimiento Pendiente' aunque el paciente ya hubiera firmado. Era como un semaforo descompuesto que siempre estaba en rojo. Ahora consulta directamente la fuente de verdad y muestra el color correcto.*

Tecnicamente: el componente **ConsentRequiredBanner.jsx** dejo de leer un campo obsoleto y pasa a consultar la funcion RPC **get_patient_consent_status**. Si el paciente ya firmo (en su portal), el banner desaparece. Si no, el banner aparece con un mensaje informativo que indica que el paciente debe firmar desde su propio portal. El comportamiento es conservador: si la funcion falla o no devuelve dato, se asume 'no firmado' para no ocultar el aviso por error.

2.7 Eliminacion de codigo deshonesto (escrituras silentes)

***Analogia:** Imagina un cajero automatico que dice 'Transaccion exitosa' pero no entrega el dinero. Es peor que un error: te enganza. La app tenia varias secciones asi: el paciente apretaba 'Guardar contacto de emergencia' y veia toast verde, pero los datos no se guardaban (porque las nuevas reglas de seguridad no le permitian escribir en esa tabla). Ahora si la operacion no funciona, te lo*

dice claramente. Si funciona, funciona de verdad.

Tecnicamente: se elimino la escritura silente que **handlePersonalSave** hacia a la tabla patients (la nueva politica RLS rechazaba pero el codigo no chequeaba). Se agrego validacion de filas afectadas (**.select('id') + data.length > 0**) en el guardado de profiles. Se elimino el handler completo de **handleEmergencySave**. Se limpio el flujo de **signConsent** en el hook del paciente para no intentar escribir donde no puede.

3. Lo que queda pendiente, ordenado por urgencia

Esta seccion es deuda real, registrada en la conversacion. Esta dividida por categoria y tamano de bloque (cuanto tiempo tomara abordarlo).

3.1 Bugs urgentes y pequenos (alta prioridad, costo bajo)

Bug del replaceState en odontograma

Cuando un dentista crea una nueva evaluacion de odontograma, el codigo cambia la URL del navegador con **window.history.replaceState** en lugar de la herramienta del router (**navigate()**). Resultado: el sistema piensa que sigues en la pagina anterior, y la auditoria de 'apertura' no se dispara hasta que refresques o navegues de nuevo. **El primer uso real no queda grabado en el log.**

Costo de fix: 1 linea de codigo (cambiar replaceState por navigate). Tiempo: 5 minutos. Impacto: cierra correctamente la auditoria del modulo odontograma.

Routing roto del modulo Odontograma

El boton 'Odontograma' del menu lateral y el boton 'Nueva Evaluacion' apuntan a la URL **/dashboard/odontograma**, pero el router declara la ruta como **/dashboard/therapist/odontograma**. Resultado: 404. El usuario solo llega al modulo escribiendo manualmente la URL correcta.

Costo de fix: 2-3 lineas (corregir los enlaces). Tiempo: 10 minutos.

3.2 Schema drift (la base de datos y el codigo no estan alineados)

***Analogia:** Es como si en una receta dijera 'agregar 2 manzanas verdes' y en el supermercado solo tuvieran rojas. El codigo busca columnas que no existen (como **therapist_services.price** o **session_activities.patient_id**) y la base de datos contesta error 400. La app sigue funcionando en lo principal, pero hay funcionalidades secundarias rotas (presupuesto del wizard de odontograma, dashboard del paciente con avisos vacios).*

Requiere migracion SQL para alinear schema. Bloque chico-medio (1-2 horas).

3.3 Compliance Ley 21.719 - Datos personales (BLOQUE GRANDE)

***Analogia:** La Ley 21.719 (Chile, 2024) dice que toda persona tiene derecho a saber que datos suyos tienes, pedir corregirlos, pedir borrarlos (derecho al olvido), y dar/retirar su consentimiento explicito para usos especificos. Hoy la app tiene la base, pero NO tiene los flujos completos para cumplir esto. Es como tener un edificio con escaleras pero sin la rampa para discapacitados que la ley exige.*

Que falta tecnicamente:

- Tabla **consent_records** y flujo para registrar consentimientos explicitos con **processing_lawful_basis** (base juridica del tratamiento de datos).
- Tabla **data_requests** y UI para que el paciente pueda solicitar acceso a sus datos (derecho ARCO: Acceso, Rectificacion, Cancelacion, Oposicion), y para que la clinica pueda responder dentro del

plazo legal (15 días hábiles).

- Notificación al paciente cuando hay tratamiento nuevo de sus datos.
- Funcionalidad de exportación completa de datos del paciente (formato portable).
- Funcionalidad de borrado de cuenta (soft delete con retención legal).

Costo: bloque grande, varias semanas. Es un frente que requiere decisiones de producto + asesoría legal antes de implementar. Es lo más crítico desde el punto de vista regulatorio para comercialización.

3.4 Hardening adicional de roles

Otros roles aún tienen permisos amplios que conviene reducir:

- **assistant** (asistente clínico): hoy tiene UPDATE amplio sobre patients. Hardening análogo al que se hizo con patient (1-2 horas).
- **clinic_admin**: definir scope exacto. Hoy está poco refinado (~2 horas).
- Fallback legacy **therapist_id** en algunos SELECT clínicos: deuda de migración al modelo nuevo de care_team. Bloque dedicado.

3.5 Funcionalidades nuevas y UI faltante

- **UI de exceptional_access_grants**: el modelo ya existe (un dentista puede pedir acceso a la ficha de un paciente que no es suyo, con autorización temporal), pero no hay pantalla para gestionarlo visualmente.
- **accept_referral** y **associate_patient_to_therapist**: flujos de derivación entre dentistas, parcialmente implementados.
- Auditoría de **documentos, impresión, exportación**: hoy solo se audita apertura de ficha y odontograma. Cuando se agreguen esas funcionalidades, hay que instrumentarlas.

4. Tus 3 preguntas, respondidas

Pregunta 1: ?Una vez terminada la app, puedo modificar visuales del frontend y crear nuevos botones para mayor funcionalidad sin tener que reconstruir?

Respuesta corta:

Si, salvo en casos especificos. Cambiar colores, textos, agregar botones, mover secciones, todo eso se hace tocando archivos del frontend (React + Tailwind CSS). No necesitas 'reconstruir' la app desde cero ni perder los datos. Cada cambio se publica con un nuevo build (compilacion) que tarda ~20 segundos.

La distincion importante:

Hay que distinguir dos sentidos de 'reconstruir':

- **Build (compilar):** esto si pasa cada vez que cambias frontend. Es automatico y rapido. Es como apretar 'guardar como PDF' en Word. No es traumatico.
- **Reconstruir desde cero (recrear la app):** NO. La estructura existe, los datos siguen, los usuarios siguen, las migraciones de BD siguen. Solo cambias el codigo del frontend.

Casos faciles (solo frontend, ~30 minutos a 2 horas):

- Cambiar colores, fuentes, bordes, tamanos.
- Agregar un boton que abre un modal nuevo con texto.
- Mover secciones, cambiar el orden de tabs.
- Agregar mensajes informativos.
- Cambiar logos, iconos.

Casos medios (frontend + posiblemente backend, 2-8 horas):

- Boton que dispara una accion en la base de datos (ejemplo: 'Marcar paciente como prioritario'). Requiere: nuevo campo en BD + nueva regla de seguridad RLS + frontend.
- Reportes nuevos (un grafico de citas por mes).

Casos grandes (refactor o feature mayor, dias o semanas):

- Agregar un nuevo modulo completo (ej: agenda integrada con WhatsApp).
- Cambiar la estructura de la BD por completo.
- Agregar un nuevo rol con permisos especificos en todo el sistema.

Lo importante a tener claro:

El frontend es la cara visible: cambiarlo NO afecta los datos. Pero **todo cambio que guarde algo nuevo en la base de datos** requiere tambien tocar la base (RLS, columnas, polices) y eso es el punto donde

hay que tener cuidado de no romper la seguridad. Por eso el flujo correcto es: planear primero, hacer cambios pequenos, validar antes de seguir.

Pregunta 2: ¿Es factible un flujo de telemedicina con foto, marcar dolores, subir radiografía y diagnostico presunto orientativo?

Respuesta corta:

Técnicamente: si, todo esto es posible. Las piezas técnicas están disponibles y son viables.
Legalmente y éticamente: hay restricciones serias que hay que respetar para no exponerse a riesgos.

Lo técnico (lo fácil):

- **Foto desde celular:** trivial. HTML estándar tiene un input `<input type='file' accept='image/*' capture='environment'>` que abre la cámara trasera del celular. Es 1 línea de código.
- **Marcar dolores en la foto o en un esquema:** componente de canvas/SVG donde el paciente dibuja o pone puntos. Hay librerías listas (react-image-annotate, fabric.js). Tiempo: 1-2 días.
- **Subir radiografía:** archivo a Supabase Storage (donde ya guardas otras cosas). Tiempo: 1 día (incluyendo validación de tipos de archivo y tamaño).
- **IA para sugerir orientación:** usar Claude API o OpenAI (GPT-4 Vision) para analizar la imagen + el texto del paciente y devolver una sugerencia. Costo por análisis: aproximadamente USD 0.01 a 0.05 por imagen. Tiempo de integración: 2-3 días.

Lo legal y ético (lo difícil):

En Chile, el diagnóstico médico/odontológico es un acto profesional reservado por ley (Código Sanitario, artículos relacionados con ejercicio profesional). Una IA NO puede emitir diagnóstico vinculante. Si la app dijera 'tu diagnóstico es: caries en muela del juicio', estaría potencialmente incurriendo en práctica ilegal de la odontología. El Colegio de Cirujano Dentistas puede objetar.

Como lo hacen Doctoralia, Médico24x7, etc.:

- Lo presentan como '**orientación preliminar**' o '**triaje no diagnóstico**'. Nunca dicen 'tienes X enfermedad'. Dicen 'lo que describes podría estar relacionado con A, B o C; agenda con un profesional para evaluación definitiva'.
- **Disclaimer obligatorio** en cada pantalla: 'Esta orientación no reemplaza la consulta presencial con un profesional. No se constituye como diagnóstico médico.'
- **Botón de derivación obligatoria:** el resultado de la IA siempre incluye 'Agenda consulta con un dentista'.
- **El profesional revisa:** antes de que el paciente vea el resultado, un dentista real lo revisa y firma. La IA es asistente, no reemplazo.

Lo que además hay que cuidar (datos sensibles):

- Las fotos y radiografías son **datos personales sensibles** bajo Ley 21.719. Requieren: almacenamiento encriptado, control de acceso estricto, logs de quien las vio (auditoría), consentimiento explícito del paciente para el tratamiento de esos datos por parte de la IA.

- **Los proveedores de IA (OpenAI, Anthropic) son destinatarios de datos sensibles.** Hay que firmar acuerdos de procesamiento de datos (DPA) con ellos y informarlo al paciente. Anthropic ofrece DPA. OpenAI también.
- Si la 'orientación' lleva al paciente a inacción y empeora una urgencia, hay riesgo de responsabilidad civil. Necesita seguro de responsabilidad civil profesional adaptado y políticas claras de uso aceptable.

Recomendación concreta:

Es factible y comercialmente atractivo si se presenta correctamente. El tiempo realista para un MVP funcional: **3 a 4 semanas de desarrollo** (no 1 día). Fases sugeridas:

- Semana 1: foto + marcar zonas de dolor + subida de archivo + tabla nueva en BD.
- Semana 2: integración con Claude API o GPT-4 Vision. Definición de prompt con disclaimer.
- Semana 3: flujo de revisión profesional (un dentista valida antes de mostrar al paciente).
- Semana 4: términos legales, consentimiento explícito, disclaimer en UI, pruebas con abogado, lanzamiento beta.

Riesgo legal si se hace bien: medio. Riesgo legal si se mal-presenta como 'diagnóstico': alto. La diferencia está en el copy, los disclaimers y la presencia de revisión profesional. Recomendando asesoría legal antes de lanzar.

Pregunta 3: ¿Es factible agregar pasarela de pago para que los pacientes paguen tratamientos?

Respuesta corta:

Si, completamente factible y relativamente directo en Chile. La tabla **payments** ya existe en el schema. Solo falta integrar una pasarela y conectar el flujo de UI.

Opciones concretas en Chile:

Pasarela	Que cubre	Tiempo	Recomendado para
Flow.cl	Webpay + multiples metodos (Mach, Multicaja, etc.)	2-4 dias	MVP rapido, multimetodo
Webpay (Transbank)	Tarjetas chilenas (estandar bancario)	~1 semana	Volumen alto, marca confiable
Mercado Pago	Tarjetas + saldo en cuenta MP	2-3 dias	Si tienes ya cuenta MP
Khipu	Transferencia electronica directa	2-3 dias	Si quieres bajos costos por transaccion

Como se integra (a grandes rasgos):

- **Frontend:** en la ficha del paciente, agregar boton 'Pagar tratamiento' con monto y descripcion. Al hacer click, redirige a la pasarela elegida.
- **Backend (Edge Function en Supabase):** recibe el request, crea una orden de pago con la pasarela (API REST), devuelve la URL para redirigir al paciente.
- **Webhook:** la pasarela notifica al servidor cuando el pago se completa o falla. El servidor actualiza la tabla **payments** con el estado real.
- **UI de confirmacion:** el paciente vuelve a la app y ve si el pago fue exitoso.

Lo que cubre por ti la pasarela (no tienes que hacerlo):

- **PCI-DSS:** las pasarelas certificadas manejan los datos de tarjeta directamente. Tu app NUNCA toca el numero de tarjeta del paciente.
- **Conciliacion bancaria automatica:** los proveedores chilenos depositan en tu cuenta (o cuenta de la clinica) automaticamente.
- **Manejo de fraudes basico:** scoring antifraude integrado.

Lo que falta integrar aparte (si emites boletas/facturas):

- **SII (boletas/facturas electronicas):** si la clinica debe emitir documentos tributarios, se integra via API SII directa o usando proveedor (Bsale, Defontana, Toteat, OpenFactura). Tiempo adicional: 2-3 dias.

Recomendacion concreta:

Empezar con Flow.cl. Es el mas simple, soporta todos los metodos relevantes en Chile (Webpay, Mach, Multicaja, transferencia), y la documentacion para developers es decente. MVP funcional: **4 a 5 dias de desarrollo**. Una vez probado, si el volumen lo justifica, se puede migrar a Webpay directo (mejor margen, pero mas trabajo).

Costos aproximados:

- **Comision por transaccion:** 2.95% a 4.5% segun pasarela y metodo. Webpay directo es el mas barato (~1.5%) pero tiene fee mensual fijo.
- **Sin costo fijo** para empezar con Flow.cl, Mercado Pago, Khipu.

5. Cierre y siguiente bloque sugerido

Si fueras a continuar trabajando en Dentalspot, mi recomendación concreta de orden:

Sesion corta (~1 hora):

- Fix del bug del `replaceState` en odontograma + fix del routing del modulo. Cierra la auditoria que ya quedo validada y deja el modulo operativo para usuarios reales.

Sesion media (~4 horas):

- Resolver schema drift de `therapist_services` y `session_activities`. Limpia errores operativos del wizard de odontograma y del dashboard del paciente.

Sesion larga (~1 semana):

- Integrar pasarela de pago (Flow.cl). Es la funcionalidad de mayor impacto comercial para clinicas reales.

Frente grande (~2-4 semanas):

- Compliance Ley 21.719 (`consent_records` + `data_requests` + ARCO). Es lo mas importante para comercialización legal.

Frente nuevo (~3-4 semanas):

- Telemedicina/triage (foto + IA + revision profesional + disclaimer legal). Es el diferenciador comercial mas atractivo, pero con riesgos legales que requieren cuidado.

Estado al cierre de las sesiones:

12 commits dedicados pusheados a origin/main, 3 migraciones SQL aplicadas a remote, auditoria clinica validada 6/6 con evidencia funcional real, frente de consentimiento consolidado, hardening del rol patient cerrado, banner del dentista alineado con fuente real, modal del dentista convertido a informativo, escrituras silenciosas del portal patient eliminadas. Branch limpio sin reescritura de historia. La app esta lista para siguiente fase.